

Реализуемый вручную (Manually implemented list)

[Списочный метод](#), реализуемый вручную, следует использовать в том случае, если возможности [декларативного метода](#) не позволяют решить поставленную перед вами задачу.

Для работы Manually implemented list в зависимости от типа реализации (см. [Method type](#)) необходимо, чтобы в состав сервиса приложения были подключены следующие [сервисные модули](#):

- при реализации метода на C++ (Native function) — сервисный модуль [BL Core](#).
- при реализации метода на Python — сервисный модуль "BL Python".
- при реализации метода на SQL Request — сервисный модуль "Бизнес-логика".

Перечисленные сервисные модули поставляются в составе [SBIS SDK](#).

Входные параметры

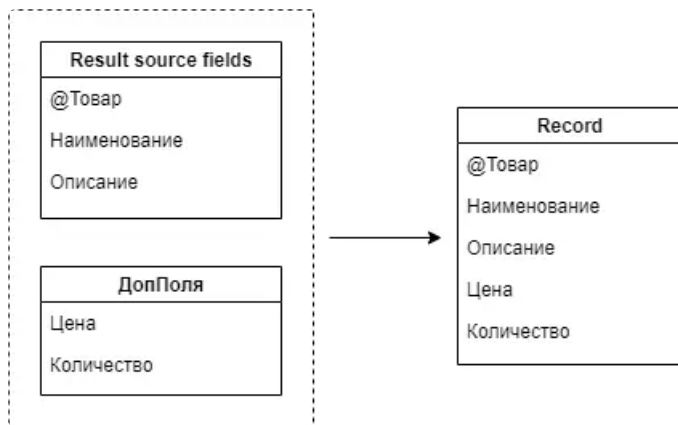
ДопПоля (ExtraFields)

Набор полей, которыми будут расширены возвращаемые записи.

Разработчик вправе по умолчанию возвращать из метода ограниченный список полей. Таким образом метод будет выполнен быстрее.

В сценариях, где набор возвращаемых полей требуется расширить, следует использовать входной параметр **ДопПоля**. Благодаря такому подходу исключается дублирование методов, отличающихся только набором возвращаемых полей.

Рассмотрим пример. По умолчанию метод возвращает три поля: "@Товар", "Наименование" и "Описание". В параметре **ДопПоля** установлено 2 поля: "Цена" и "Количество". В итоге записи выборки будут состоять из 5 полей: "@Товар", "Наименование", "Описание", "Цена" и "Количество".



Фильтр (Filter)

Параметры фильтрации выборки.

Значения параметров могут быть использованы при создании условий фильтрации. Без использования в условиях фильтрации никак не влияют на результаты выборки.

Сортировка (Sorting)

Параметры сортировки записей в выборке.

Сортировка на стороне бизнес-логики представлена классом [SortingList](#), который является массивом [ColumnSortParameters](#).

Для конфигурации параметра сортировки по полю применяют следующие опции:

- **fieldName** — имя поля, по которому производится сортировка.
- **order** – порядок сортировки. Возможные значения:

- `true` — обратный порядок сортировки (по убыванию, DESC);
- `false` — прямой порядок сортировки (по возрастанию/ алфавитный порядок, ASC).
- **nullPolicy** – политика обращения с NULL-значениями. Возможные значения:
 - `true` — записи расположены в конце сортируемой последовательности;
 - `false` — записи с NULL-значениями расположены в начале сортируемой последовательности.

Навигация (Pagination)

Параметры для навигации с типом [постраничная](#) или [по курсору](#).

Навигация на стороне бизнес-логики представлена классом [Navigation](#).

Опции:

- **Limit** — число записей на странице.
- **Page** — номер страницы, записи которой будут возвращены декларативным методом. Нумерация страниц начинается с нуля.
- **HasMore** — признак наличия следующих страниц выборки. Возможные значения:
 - `true` — в ответе от сервера бизнес-логики будет находиться признак: есть следующие страницы.
 - `false` — в ответе от сервера бизнес-логики возвращается общее количество записей выборки.

Если данный параметр не был передан, то сервер бизнес-логики не возвращает никаких дополнительных данных об общем количестве записей.
- **Position** — курсор. Применяется для списков с [навигацией по курсору](#).
- **Direction** — направление выборки относительно курсора. Возможные значения:
 - `bothways`,
 - `forward`,
 - `backward`.

Пример:

```

1  ...
2  .query(new
    source.Query().where(filter).limit(100).offset(5*100).addErrback(function(e) {
3      console.error(e);
4  }));

```

Опции, настраиваемые через интерфейс Genie

Name

[См. описание](#)

Alias

[См. описание](#)

Comment

[См. описание](#)

Accessibility

[См. описание](#)

Frequent call protection

[См. описание.](#)

Don't log method arguments

[См. описание.](#)

Enable cache

[См. описание.](#)

Enable http cache

[См. описание.](#)

Access areas

[См. описание.](#)

Handler

Обработчик — вспомогательный метод БЛ, который можно вызвать до или после выполнения списочного метода. Подробнее читайте [здесь](#).

Has call timeout

Время ожидания выполнения метода. Применяется в случаях, когда таймаут явно не задан.

Если указан таймаут — 1 минута, а метод отработал за 10 минут, то на вызывающей стороне мы получим ошибку уже через 1 минуту.

Has execution timeout

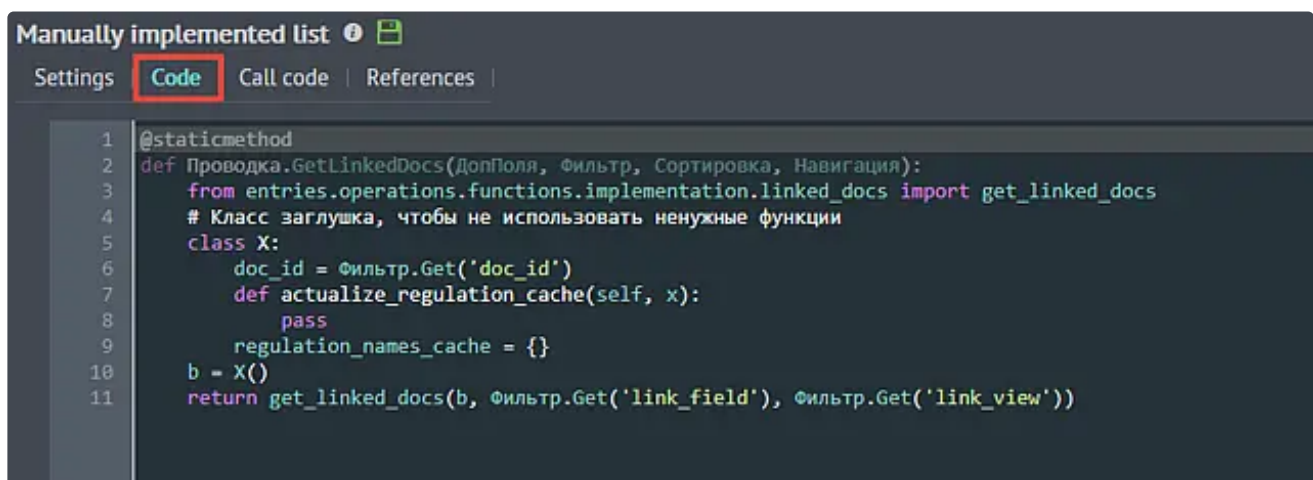
Таймаут (время) выполнения метода. Если указан таймаут — 1 минута, то через 1 минуту воркер будет прерван при помощи биения.

В случае прерывания воркера мы получим ошибки в логах. Применяется при возникновении нештатных ситуаций.

Method type

Тип реализации метода. Определяет язык программирования (Python, SQL или C++ (Native function)), на котором будет разработан метод.

Создание метода с реализацией на Python или SQL производится в Genie во вкладке Code:



```
Manually implemented list ⓘ 📄
Settings | Code | Call code | References |
1 @staticmethod
2 def Проводка.GetLinkedDocs(ДопПоля, Фильтр, Сортировка, Навигация):
3     from entries.operations.functions.implementation.linked_docs import get_linked_docs
4     # Класс заглушка, чтобы не использовать ненужные функции
5     class X:
6         doc_id = Фильтр.Get('doc_id')
7         def actualize_regulation_cache(self, x):
8             pass
9         regulation_names_cache = {}
10    b = X()
11    return get_linked_docs(b, Фильтр.Get('link_field'), Фильтр.Get('link_view'))
```

Способ реализации метода на C++ более трудоёмкий и состоит из следующих этапов:

- Создание функции в исходном коде. Имена функции (в исходном коде) и списочного метода (для которого выбрана реализация на C++) должны быть идентичными.
- Компиляция исходного кода в библиотеку DLL.
- Размещение библиотеки DLL в директории сервисного модуля.

Return values

Тип значения, которое возвращает метод БЛ. Из исходного кода метода разработчик должен вернуть именно такой тип.

Если из кода возвращается тип Record, тогда в опции укажите Record.

Если из кода возвращается тип RecordSet, тогда в опции укажите Table.

Если из кода возвращается массив значений, тогда в опции укажите Array.

Если возвращаются любые другие значения, тогда в опции укажите Scalar.

Если метод ничего не должен возвращать, тогда укажите значение None.

Если тип возвращаемого значения отличается от значения, что установлено в этой опции, тогда вызов метода завершится с ошибкой.

Filter parameters

Дополнительные параметры, которые передаются при вызове метода во входном параметре [Фильтр \(Filter\)](#).

Предполагается, что такие параметры будут использованы при создании условий фильтрации в исходном коде метода.

А в исходном коде метода этот параметр можно использовать через переменную Фильтр.

Обращение к опциональному параметру нужно делать через метод TestField:

```
1 ...
2 if Фильтр.TestField('Ид0') is not None:
3 ...
```

Обращение к обязательному параметру можно задать через точку:

```
1 ...
2 if Фильтр.Раздел:
3 ...
```

Readonly

Обозначить метод как "на чтение". Используется в [системе прав](#) для предоставления доступа клиенту, для которого установлены права доступа "Разрешено только чтение".

- Для каждого [участка системы](#), включаемого в [роль](#), можно настроить права доступа на его объекты доступа: объекты и методы БЛ. Права доступа бывают следующих видов: "Запрещено всё", "Разрешено только чтение" и "Разрешены чтение и изменение". Если на объекты доступа назначены права "Разрешено только чтение", то в итоге в роли пользователя будут доступны только методы с установленной опцией "на чтение".
- Если метод не вносит никаких изменений в БД, то установите эту опцию, чтобы при настройке системы прав доступа метод не был утерян для соответствующих ролей.

Transaction type

Тип транзакции метода, подробнее о котором вы можете прочитать в разделе [Управление транзакциями в сервисном фреймворке](#).

Примеры исходного кода в опции Code

Следующий пример демонстрирует реализацию списочного метода на языке Python. Кроме набора записей, в ответ включены строка итогов, хлебные крошки и результаты навигации.

```
1 def Goods.ManuallyList(ДопПоля, Фильтр, Сортировка, Навигация):
2     # Создаём формат полей результирующего списка.
3     f = RecordFormat()
4     f.AddInt64('id')
5     f.AddObjectId('hierarchy')
6     f.AddString('name')
7     f.AddMoney('sum')
8
```

```
9      # Создание результирующих данных.
10     result = RecordSet(f)
11     result.AddRow(Record({'id':10, 'hierarchy':2, 'name':'товар1',
'sum':100.15}))
12     result.AddRow(Record({'id':11, 'hierarchy':2, 'name':'товар2',
'sum':200.22}))
13
14     # Создание строки итогов.
15     result.outcome = Record({'name':'Итого', 'sum':300.37})
16
17     # Создание хлебных крошек.
18     path = RecordSet(f)
19     path.AddRow(Record({'id':1, 'hierarchy':None, 'name':'папка1'}))
20     path.AddRow(Record({'id':2, 'hierarchy':1, 'name':'папка2'}))
21     result.path = path
22
23     # Создание результатов навигации.
24     if Навигация.IsNext():
25         # Устанавливаем, что есть еще записи.
26         result.nav_result = NavigationResult(True)
27     else:
28         # Устанавливаем количество записей, подпадающих под фильтр.
29         result.nav_result = NavigationResult(100500)
30
31     # Ответ метода – RecordSet.
32     # В настройках метода должна быть установлена опция "Result type" в
значение "Table".
33     return result
```